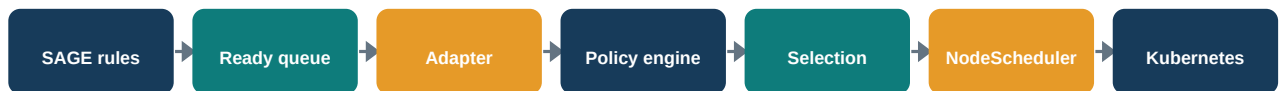


# SAGE Resilient Urgent Scheduler

## Repository guide

Architecture, behavior, integration boundaries, THOR/HALO mapping, deployment status, and current limitations.

**In one sentence: this repository is an explainable admission and scheduling policy for SAGE/Waggle edge applications. It decides which ready application should run next.**



**This repository decides which ready plugins may run.**

SAGE creates and tracks Pods; Kubernetes performs final placement.

| It does                                                                                      | It does not                                                                                            |
|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Rank and admit ready SAGE plugins using urgency, age, reliability, and resource constraints. | Run AI models, choose CNN versus DINO, control Pods directly, or implement MQTT/Meshtastic transports. |

Based on the repository contents inspected on 27 July 2026. No source code was changed while preparing this guide.

# Contents

---

- 01 Core scheduling engine
- 02 Policy configuration
- 03 SAGE/Waggle adapter
- 04 Failure handling
- 05 THOR/HALO camera orchestration prototype
- 06 Offline tools
- 07 Integration binary, packaging, and deployment
- 08 Tests and validation
- 09 Important limitations
- 10 Mapping to the target diagram

## System boundary

---

The repository adds a policy layer on top of existing SAGE/Waggle mechanisms. SAGE decides when a science rule makes an application ready. This policy ranks and admits ready applications. The existing NodeScheduler creates the Pod, and Kubernetes chooses the final node.

```
SAGE science rule -> ready queue -> adapter -> policy engine -> explained selection ->
NodeScheduler -> Kubernetes
```

**The most important boundary: the repository selects or defers work. It does not create, delete, migrate, checkpoint, or preempt Pods.**

Source: docs/architecture.md:9

## 1. Core scheduling engine

---

The core package is deterministic and independent of SAGE and Kubernetes. It accepts a snapshot of ready and running work and returns a fully explained decision.

### What already exists

- A generic Task containing identity, priority, enqueue time, estimated runtime, deadline, GPU requirement, predicted success rate, and CPU/memory/GPU resources.
- A Snapshot containing ready tasks, running tasks, the decision time, and optional available capacity.
- A Decision containing the selected tasks and one outcome for every candidate.

Source: pkg/policy/model.go:29

## Decision outcomes

| Outcome         | Meaning                                   |
|-----------------|-------------------------------------------|
| <b>selected</b> | The task is allowed to start now.         |
| <b>deferred</b> | The task is valid but cannot start yet.   |
| <b>rejected</b> | The task identity or metadata is invalid. |

Reasons include `concurrency_limit`, `gpu_capacity`, `resource_capacity`, `reliability_threshold`, `already_running`, and `invalid_task`.

Source: `pkg/policy/engine.go:11`

## How ranking works

The deadline is either supplied explicitly or calculated from enqueue time plus maximum queue latency. Slack subtracts both the current time and estimated runtime from that deadline.

```
deadline = explicit deadline
or enqueue time + maximum queue latency

slack = deadline - current time - estimated runtime
```

The score combines priority, deadline urgency, queue age, and predicted success. The example policy uses 45%, 30%, 15%, and 10% respectively. Older tasks gain age points to reduce starvation. Tasks close to or past their deadline gain urgency.

An impossible deadline is marked with `predictedDeadlineMiss`, but the task may still be selected because the engine does not know an alternative application.

Ties are deterministic: earliest deadline, then oldest enqueue time, then lexical task ID.

Source: `pkg/policy/engine.go:90`

## How admission works

- Validate task identity, metadata, and uniqueness.
- Defer a task that is already running.
- Apply the minimum success-rate threshold.
- Check the global concurrency limit.
- Check the GPU concurrency limit.
- Check CPU, memory, and GPU fit when resource fitting is enabled.

Source: `pkg/policy/engine.go:127`

## 2. Policy configuration

Configuration is strict, versioned YAML. It contains policy behavior only, keeping the engine portable and deterministic.

- Global and GPU concurrency limits.
- Default priority, runtime, maximum queue latency, and success rate.
- Minimum acceptable success rate and configurable score weights.
- Optional resource-fit enforcement.

- A trust switch for workload-provided environment hints.
- Fail-open behavior for internal policy errors.

The loader rejects unknown fields, missing schema headers, multiple YAML documents, invalid durations or probabilities, excessive concurrency, and invalid weights.

**The supplied policy is intentionally conservative: one application at a time, one GPU workload at a time, resource fitting disabled, workload hints disabled, and fail-open enabled.**

Source: pkg/policy/config.go:28

Source: config/policy.example.yaml:1

### 3. SAGE/Waggle adapter

The adapter converts Waggle PluginRuntime objects into generic policy tasks, executes the engine, and returns only pointers that originated in the ready queue.

- Snapshots the ready and scheduled queues.
- Validates Kubernetes names and resource quantities.
- Tracks first-observed time in memory so queue age can be scored.
- Preserves the original queue pointers and never pushes, pops, or reorders entries.
- Stores an independent copy of the latest explained decision.

The upstream NodeScheduler remains responsible for removing selected plugins, moving them to the scheduled queue, creating Pods, and tracking lifecycle events.

Source: integrations/sage/adapter/policy.go:90

#### Workload scheduling hints

| Variable                         | Purpose                          |
|----------------------------------|----------------------------------|
| SAGE_SCHEDULER_PRIORITY          | Priority from 0 to 100           |
| SAGE_SCHEDULER_MAX_LATENCY       | Maximum acceptable queue wait    |
| SAGE_SCHEDULER_ESTIMATED_RUNTIME | Expected task duration           |
| SAGE_SCHEDULER_DEADLINE_AT       | Absolute RFC3339 deadline        |
| SAGE_SCHEDULER_SUCCESS_RATE      | Declared probability from 0 to 1 |

**These values are declarations from the application author, not measurements. With `trustWorkloadEnvHints` set to `false`, they are ignored and the policy defaults are used.**

Source: integrations/sage/adapter/hints.go:13

#### GPU and resources

A task is classified as GPU work through `selector.resource.gpu`, `limit.gpu`, or `nvidia.com/gpu`. CPU, memory, GPU, and extended-resource quantities are validated. Requests cannot exceed limits.

Real resource fitting remains disabled because the pinned Waggle revision supplies fictitious availability. Kubernetes still performs the final resource enforcement.

Source: docs/sage-integration.md:103

## 4. Failure handling

Invalid tasks are rejected individually. Internal policy errors can either fail closed or use a bounded fail-open path.

- Fail-open selects valid work in queue order.
- Global and GPU limits remain enforced.
- Duplicate identities and unsafe same-name collisions remain blocked.
- Invalid hints do not automatically trigger the fallback.

**Fail-open means: policy engine failed -> continue conservatively using FIFO. It does not mean: DINO failed -> launch CNN.**

Source: `integrations/sage/adapters/policy.go:358`

## 5. THOR/HALO camera orchestration prototype

**The current working tree contains an untracked pkg/orchestration prototype. It is not yet part of the committed or documented scheduler MVP.**

### What already exists

- A generic interface for directly connected cameras.
- A versioned `halo.capture/v1` request and response contract.
- A THOR-side adapter for a remote HALO camera.
- A HALO-side camera endpoint.
- Concurrent capture of THOR and HALO images using one request ID.
- Validation of correlation, camera identity, media type, image size, timestamp, and capture skew.
- Serialization so two sessions cannot interleave work on the same physical camera pair.
- A generic PairAnalyzer boundary receiving the validated image pair.

Defaults are a 15-second capture timeout, 2-second maximum timestamp skew, and 8 MiB maximum image size.

Source: `pkg/orchestration/types.go:14`

Source: `pkg/orchestration/coordinator.go:29`

Source: `pkg/orchestration/halo.go:78`

### What the prototype does not contain

- A concrete camera driver.
- MQTT, Meshtastic, image fragmentation, or WAN transport.
- Mortimus or another smoke-detection application.
- CNN, DINO, JEPA, or another analyzer implementation.
- Analyzer results, smoke scores, or alert publication.
- Any integration with the scheduler binary.

It establishes a clean camera-pairing boundary. Concrete transport and AI implementations still need to plug into its interfaces.

## 6. Offline tools

### policyctl

policyctl validates policy YAML, reads a strict JSON snapshot, executes the engine, and emits an explained JSON decision. It has no network client and can be used without SAGE or Kubernetes.

Source: `cmd/policyctl/main.go:21`

### chaoslab

chaoslab performs deterministic sensitivity analysis. It can perturb priority, arrival time, deadline, estimated runtime, maximum queue latency, success rate, and available resources. It compares selected sets with Jaccard distance.

It studies decision boundaries offline. It does not inject failures into Kubernetes or physical hardware.

Source: `pkg/chaos/experiment.go:13`

### Examples

- A ready-task snapshot comparing an urgent GPU smoke detector with a routine image sampler.
- A sensitivity experiment that changes deadline, arrival, reliability, and runtime.
- A SAGE job with urgent and routine plugins plus environment hints.

Source: `examples/snapshots/urgent-vs-routine.json:1`

Source: `examples/chaos/sensitivity.json:1`

Source: `examples/sage/job.example.yaml:1`

## 7. Integration binary, packaging, and deployment

### SAGE integration binary

The integration module pins a specific Waggle edge-scheduler commit and builds a replacement `sage-nodescheduler` executable. It constructs the upstream `NodeScheduler` and injects this policy into its `SchedulingPolicy` field.

- Loads strict Waggle and policy configuration.
- Provides an offline `--validate-config` mode.
- Rejects unsafe live simulation and no-RabbitMQ modes.
- Rejects empty or inherited default RabbitMQ credentials unless explicitly acknowledged.

**Documented status: prepared but not deployed or connected to a SAGE node. Validation mode is the only intended mode until deployment is confirmed.**

Source: `integrations/sage/cmd/waggle-nodescheduler/main.go:33`

Source: `README.md:7`

### Container image

The Dockerfile builds a static binary and places it in a minimal distroless non-root image. It includes the policy and `NodeScheduler` example configurations.

Source: Dockerfile:1

## Prepared Kubernetes manifests

An example ConfigMap and Deployment patch are provided, together with controlled restart instructions. The repository never applies them automatically.

Before deployment, operators must confirm the real namespace, Deployment and container names, SAGE/Waggle commit, k3s version, target architectures, service account, image digest, RabbitMQ secrets, and network exposure.

Source: integrations/sage/deploy/README.md:1

## 8. Tests and validation

The repository includes policy, adapter, CLI, chaos, integration, and optional control-plane tests.

- Ranking, deadlines, aging, reliability, concurrency, and resource admission tests.
- Determinism, input-permutation, aliasing, and fuzz tests.
- Strict configuration and resource parser tests.
- Queue identity, pointer preservation, collision, fail-open, and fail-closed tests.
- CLI and offline sensitivity tests.
- Camera-pair correlation, skew, target, and session-serialization tests in the untracked prototype.

KWOK fixtures simulate CPU and GPU nodes and verify Kubernetes placement. They do not run containers, physical GPUs, cameras, RabbitMQ, or the scheduler process.

Source: test/kwok/README.md:1

The Makefile combines formatting, tests, race tests, vetting, builds, configuration validation, and examples.

Source: Makefile:8

**Verification note: the test suite could not be executed during this inspection because the current shell did not contain the go executable.**

## 9. Important limitations

| Area                  | Current limitation                                                        |
|-----------------------|---------------------------------------------------------------------------|
| <b>Queue identity</b> | Some upstream removals compare plugin name instead of full identity.      |
| <b>Queue snapshot</b> | The pinned queue does not expose an atomic snapshot.                      |
| <b>Local REST API</b> | The pinned schedule endpoint creates incomplete runtime objects.          |
| <b>Pod failure</b>    | Upstream Pod-creation failures are not always requeued.                   |
| <b>State</b>          | Queue age is in memory and resets after restart.                          |
| <b>Resources</b>      | Available capacity is not trustworthy, so fit enforcement is disabled.    |
| <b>Observability</b>  | No NodeScheduler Prometheus metrics or runtime success feedback.          |
| <b>Resilience</b>     | No preemption, retry budget, checkpointing, replication, or migration.    |
| <b>AI models</b>      | No model degradation, drift detection, or alternative-model selection.    |
| <b>Licensing</b>      | The pinned Waggle dependency must be reviewed before binary distribution. |

Source: docs/sage-integration.md:128

Source: NOTICE:7



## 10. Mapping to the target diagram

| Diagram component         | Repository status                                               |
|---------------------------|-----------------------------------------------------------------|
| SAGE scheduler            | Implemented as the generic urgent and resilient policy.         |
| THOR to HALO capture      | Partially represented by the untracked orchestration prototype. |
| Paired-image validation   | Implemented in the prototype.                                   |
| MQTT / Meshtastic         | Not implemented.                                                |
| Mortimus                  | Not implemented.                                                |
| CNN / DINO / JEPA         | No inference implementations and no model selection.            |
| WAN-THOR / Beehive alerts | Not implemented by this repository.                             |
| SAGE deployment           | Prepared but not performed.                                     |

**Bottom line: the repository provides the scheduling brain, an offline policy laboratory, a SAGE adapter, and an uncommitted camera-pair orchestration boundary. The transport, smoke-detection application, model cascade, and operational deployment remain separate work.**

## Reference root

```
/Users/clementnunes/projects/sage-resilient-urgent-scheduler
```

Repository-specific code is Apache-2.0. The NOTICE file explains that the pinned Waggle edge-scheduler dependency requires a separate license review before distributing a binary.